

Parcial PRO2

1. Clases y orientación a objetos (Opcional)

- **Classes:** Convierten **structs** en TADS con privacidad (**public:** y **private:**) para asegurar la consistencia de los datos.
- **Constructor:** Puntos de inicio con el mismo nombre que la **class**. **Sobrecarga:** **Class()**, **Class(const Class& other)**.
- **Lista de Inicializadores:** **: v1(0), v2(b) {}** Para referencias e inicializar objetos miembro de golpe.
- **const methods:** **void get() const;** - Método que no modifica los atributos propios.
- **static methods:** Métodos aplicables a nivel global desde la clase. **Classe::metode();**.
- **Inmersiones:** **inc_()**. **Atributos privados:** **int a_.**
- **Control de Ficheros (.hh/.cc) y Make:**
 - **.hh:** 1º) **#ifndef XXXX_HH**, 2º) **#define XXXX_HH**, 3º) **class XXXX { ... };** 4º) **#endif**.
 - **.cc:** **#include "XXXX.hh"**. Se aplica **XXXX::XXXX(...)** { **a_ = a;** }.
 - **Makefile:** **XXXX.o: XXXX.cc XXXX.hh** debajo **g++ -c XXXX.cc**.
- **inline:** Elimina el coste de la llamada a la función al integrarla en tiempo de compilación donde toque.
- **assert:** **#include «assert.hh»**. Detiene automáticamente el programa si no se cumplen las precondiciones.

2. Piles y colas

- **Stack / Queue / PQ:** **#include <stack> / #include <queue>**. **pop()** siempre retorna **void**. Para obtener el valor hace falta hacer **top()** o **front()** antes del **pop()**.
- **Stack (LIFO):** **push(x), pop(), top(), empty(), size()**.
- **Queue (FIFO):** **push(x), pop(), front(), empty(), size()**.

3. Llistes (List) e Iteradores

- **list<T> / deque<T>:** **#include <list> / #include <deque>**.
 - **Métodos comunes:** **push_back(x), push_front(x), pop_back(), pop_front(), insert(it, x), erase(it), back(), front()**.
 - *Diferencia:* **deque** tiene **operator[]** ($O(1)$), **list** NO.
- **vector<T>:** **#include <vector>**.
 - **Métodos:** **push_back(x), pop_back(), insert(it, x), erase(it), back(), front(), operator[]**.
- **Iteradores:**
 - **iterator:** Lectura y escritura. **list<T>::iterator it**
 - **const_iterator:** Solo lectura. **list<T>::const_iterator**
 - **reverse_iterator:** Inverso, lectura y escritura. **list<T>::reverse_iterator**
 - **const_reverse_iterator:** Inverso, solo lectura. **list<T>::const_reverse_iterator**
 - **Posiciones:** **begin(), end(), rbegin(), rend()**.
 - **Uso:** ***it** (acceso), **it++**, **it--**. Devuelve **it = L.erase(it);**.

4. Árboles binarios (BinTree<T>)

- **BinTree<T>:** **#include "BinTree.hh"**. Estructura recursiva inmutable.

- Métodos: `value()`, `left()`, `right()`, `empty()`.
- **Recorridos:** DFS (Pre/In/Postorden) y BFS (niveles).
- **Estrategias de Resolución:**
 1. **Casos base:** Comprobar `t.empty()` y, si es necesario, caso **hoja** (`left` y `right` vacíos).
 2. **Inmersión:** Por si los árboles hablan entre ellos (Ej: si todos los nodos son iguales, si todas las hojas son iguales, si la suma de nodos hijos = nodo actual, etc). Usar función auxiliar (`nombre_()`) con parámetro por **referencia** (ej: `int& x`) para guardar estados.
 3. **Salto de fe:** Resuelve el nodo actual y asume que la recursividad ya funciona para los hijos.

5. Colas de prioridad y árboles generales

- **Heap<T>:** `#include "heap.hh".`
 - Métodos: `push(x)`, `pop()`, `top()`, `empty()`, `size()`.
- **Tree<T>:** Árbol general (n-ario).
 - Métodos: `value()`, `num_children()`, `child(i)`, `empty()`.

6. Diccionarios: map y set

- **set<T>:** `#include <set>.` **Conjunto ordenado** que no admite duplicados. Para usar **set** con un **struct**, hay que definir el **operator<** (establecer el orden).
 - Métodos: `insert(x)`, `erase(x)`, `find(x)`, `empty()`, `size()`.
 - Iteradores: `begin()`, `end()`. Se accede al valor con `*it`.
- **multiset<T>:** Igual que el **set**, pero permite duplicados (útil para rankings donde dos elementos pueden empatar en todo).
- **map<K, V>:** `#include <map>.` Ordena automáticamente por la **clave** (K). Elementos internos: `pair<clave, valor>`.
 - Métodos: `m[clave] = val`, `insert({clave, val})`, `erase(x)`, `find(clave)`, `empty()`, `size()`.
 - Iteradores: `begin()`, `end()`. Se accede como `it->first` (clave) y `it->second` (valor).

7. Rendimiento y Complejidad (Resumen)

Método	Stack/Queue	Heap (PQ)	Vector	Deque	List	Map/Set
push / insert	$O(1)$	$O(\log N)$	$O(1)^* / O(N)$	$O(1)^* / O(N)$	$O(1)$	$O(\log N)$
pop / erase	$O(1)$	$O(\log N)$	$O(1) / O(N)$	$O(1) / O(N)$	$O(1)$	$O(\log N)$
top / front / back	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	-
operator[]	-	-	$O(1)$	$O(1)$	-	$O(\log N)$
find	-	-	-	-	-	$O(\log N)$

*Coste amortizado. `insert(pos, x)` y `erase(pos)` en **list** son $O(1)$ si ya tenemos el iterador.